

一维 Poisson 方程与常微分方程的 数值求解及误差分析

庄肃涵

工程力学拔尖班

学号：3025206069

2026 年 9 月 13 日

摘要：本文围绕一维 Poisson 边值问题和一阶常微分方程初值问题，完成算法推导、MATLAB 实现、误差验证和方法比较。Poisson 方程采用二阶中心有限差分离散为稀疏三对角线性系统，并结合解析解、节点误差、截断残差和网格加密检验精度；随后利用 Richardson 外推提高结果精度。初值问题采用自编经典四阶 Runge-Kutta 方法求解，并通过手工推导和符号计算获得解析解，再与 Euler 及 Heun 型二阶 Runge-Kutta 方法比较。向量形式的 RK4 还用于线性打靶，从另一条计算路径求解同一 Poisson 问题。实验中，有限差分、Euler、RK2、RK4 和打靶法最大误差的拟合阶分别为 2.002502、1.018057、2.046956、4.036671 和 4.004176，Richardson 外推的相邻观测阶接近 4。结果与各方法的理论阶数一致；在本文采用的网格和函数调用预算下，外推与高阶方法均表现出明显的精度优势。项目最终形成了统一入口、独立求解函数以及完整数据和图表输出的 MATLAB 程序。

关键词：MATLAB；Poisson 方程；有限差分法；Runge-Kutta 法；Richardson 外推

目录

一、问题描述与总体设计	1
1.1 问题重述	1
1.2 总体求解方案	1
1.3 MATLAB 程序组织	1
1.4 实验参数与评价指标	2
二、一维 Poisson 方程的有限差分求解	3
2.1 解析解与中心差分格式	3
2.2 稀疏线性系统的 MATLAB 实现	4
2.3 数值解与空间误差分布	4
2.4 二阶收敛与残差分析	5
三、Richardson 外推与精度提升	6
3.1 主误差项消去	6
3.2 粗细网格对应与 MATLAB 实现	6
3.3 精度提升与观测阶	6
四、常微分方程初值问题的 RK4 求解	7
4.1 解析解推导与符号计算	7
4.2 经典 RK4 与向量状态实现	8
4.3 数值解与有符号误差	9
4.4 四阶收敛与归一化误差	9
五、Euler、RK2 与 RK4 的精度和效率比较	10
5.1 更新公式与统一比较接口	10
5.2 相同步长下的误差与阶数	11
5.3 函数调用次数与精度	11
5.4 实测时间与结论边界	12

六、基于 RK4 的线性打靶法与交叉验证.....	13
6.1 边值问题向初值问题的转化.....	13
6.2 MATLAB 复用与端点处理	14
6.3 三方比较与误差解释	14
七、结论	15
参考文献	16
附录 A 核心算法源代码	17
A.1 solvePoissonFD.m.....	17
A.2 poissonExact.m.....	17
A.3 richardsonExtrapolatePoisson.m	18
A.4 ivpExactSymbolic.m	18
A.5 rk4Solve.m	18
A.6 eulerSolve.m.....	19
A.7 rk2Solve.m	20
A.8 solvePoissonShooting.m.....	20
A.9 errorNorms.m	21
A.10 computeOrder.m.....	21
附录 B 补充数据与诊断图	22
附录 C 运行说明与结果索引	23

一、问题描述与总体设计

1.1 问题重述

课程第七大类第 15 题要求采用有限差分法求解区间 $[-1,1]$ 上的一维 Poisson 边值问题。本文按照题目给定的正二阶导数形式进行离散。

$$u''(x) = e^{-x^2}, -1 \leq x \leq 1, u(-1) = u(1) = 0 \quad (1.1)$$

第 16 题要求自行实现经典 RK4，并用符号计算求得解析解后进行比较。

$$y'(t) = -y(t) + t^2 + 3, y(0) = 1, 0 \leq t \leq 3 \quad (1.2)$$

下文分别将两题简称为 Poisson 问题和初值问题。

1.2 总体求解方案

本项目由工程力学拔尖班庄肃涵独立完成。研究依次开展解析解推导、程序接口设计、有限差分计算、误差分析、Richardson 外推、RK4 实现、方法比较和打靶法扩展。各阶段统一保存节点数据和汇总结果，便于后续分析直接复用。

Poisson 问题先由有限差分得到基准离散解，再在相同源项和边界条件下逐级加密网格。Richardson 外推组合粗、细网格结果以消去主要误差项；打靶法则把边值问题转化为初值问题，调用向量 RK4 积分并根据右端边界修正初始斜率。

初值问题先求解析表达式，再以 RK4 为主要算法，并用 Euler 和 RK2 作低阶对照。向量 RK4 在打靶法中的复用进一步联系了两题的求解过程。

1.3 MATLAB 程序组织

根目录 runAll.m 负责配置路径并依次执行各项实验。src/poisson 和 src/ode 分别保存边值问题与初值问题求解器，src/common 提供误差、阶数和绘图工具，src/experiments 负责实验参数及文件输出。求解函数专注于数值计算，网格设置、结果保存和绘图由实验函数统一完成。

表 1-1 主要程序接口与职责

函数	输入要点	输出与职责
solvePoissonFD	N、源项、边界、区间	x、uNum、meta；稀疏差分求解
richardsonExtrapolatePoisson	粗网格 N 及问题参数	共同节点外推解及粗细网格信息
rk4Solve / eulerSolve / rk2Solve	右端函数、区间、初值、步数	t、yNum、meta；单步推进
ivpExactSymbolic	无输入	符号解、数值函数、检验结构体

函数	输入要点	输出与职责
<code>solvePoissonShooting</code>	N 及边值问题参数	打靶解、修正斜率及诊断量
<code>errorNorms / computeOrder</code>	数值与精确值 / 步长与误差	误差范数、相邻阶与拟合阶

数值数据保存在 `results` 目录，图形保存在 `figures` 目录。CSV 文件用于汇总和制表，MAT 文件保存结构体、节点向量、符号检验信息及实验环境。正文表格均取自己保存的数据文件，并按统一精度显示。完整算法和运行方式见附录 A、C。

1.4 实验参数与评价指标

表 1-2 实验网格与保存内容

实验	区间数或步数 N	用途
有限差分基础分析	8、16、32、64、128、256	节点解、空间误差和残差
有限差分正式收敛 / 打靶	8、16、32、64、128、256、512	七级收敛及交叉比较
Richardson 外推	粗网格 8 至 256，逐次加倍	每组另用 2N 细网格
RK4 基础分析	6、12、24、48	解曲线、有符号及归一化误差
RK4 正式收敛 / 三方法比较	6、12、24、48、96、192、384	七级收敛、工作量和计时

N 表示区间等分数或时间步数，计入两个端点后共有 N+1 个节点。Poisson 问题的步长为 2/N，初值问题的步长为 3/N。节点误差定义为数值解减去解析解，整体精度采用最大绝对误差和均方根误差衡量；空间问题另计算加权离散二范数。

$$E_{\infty} = \max_{0 \leq i \leq N} |e_i|, E_{RMS} = \sqrt{\frac{1}{N+1} \sum_{i=0}^N e_i^2} \quad (1.3)$$

$$E_{2,h} = \sqrt{h \sum_{i=0}^N e_i^2} = \sqrt{h} \|e\|_2 \quad (1.4)$$

`errorNorms` 返回的 `l2Error` 是不加权的向量二范数，与连续函数的积分 L2 误差含义不同。式(1.4)另行定义了空间分析所用的加权离散量。RMS 误差反映节点误差的平均尺度，最大误差用于估计收敛阶。

$$p_i = \frac{\log \frac{E_i}{E_{i+1}}}{\log \frac{h_i}{h_{i+1}}}, \log E = \hat{p} \log h + b \quad (1.5)$$

实验环境为 MATLAB 26.1.0.3346908 (R2026a) Update 5、PCWIN64，数值计算采用双精度，符号解析解使用 Symbolic Math Toolbox。初值方法比较包含 21 个方法与步

数组组合，每组执行 7 次 `timeit`，共保存 147 条计时估计。`timeit` 会在内部重复调用目标函数，因此每条记录表示一次计时估计。

二、一维 Poisson 方程的有限差分求解

2.1 解析解与中心差分格式

源项为偶函数，且左右边界值相同，因此解析解关于原点对称。对源项两次积分，并利用误差函数的定义和端点条件，可得

$$u(x) = \frac{e^{-x^2} - e^{-1}}{2} + \frac{\sqrt{\pi}}{2} [x \operatorname{erf}(x) - \operatorname{erf}(1)] \quad (2.1)$$

对式(2.1)求导并代入端点，可验证其满足原方程和零边界条件。由于二阶导数为正，解在区间内呈凸形，内部取负值，最低点位于中心。这些性质可用于初步检查数值曲线的符号、边界和对称性，精度则由后续误差计算确定。程序 `poissonExact` 采用逐元素运算一次得到全部节点的解析值。

在均匀节点处分别作正负方向的 Taylor 展开，相加后奇次项消失。对于本算例的光滑解，二阶中心差分满足^[1]

$$\frac{u(x_i - h) - 2u(x_i) + u(x_i + h))}{h^2} = u''(x_i) + \frac{h^2}{12} u^{(4)}(x_i) + O(h^4) \quad (2.2)$$

用中心差分商代替二阶导数，可得内部节点上的离散方程。本题采用正二阶导数形式，因此矩阵主对角线为负，两条副对角线为正；矩阵与右端项必须使用同一符号约定。

$$\frac{U_{i-1} - 2U_i + U_{i+1}}{h^2} = e^{-x_i^2}, i = 1, \dots, N-1 \quad (2.3)$$

$$A = \frac{1}{h^2} \operatorname{tridiag}(1, -2, 1), AU = b \quad (2.4)$$

未知向量包含 $N-1$ 个内部节点值。左端边界对第一行右端项的修正量为 $-u(a)/h^2$ ，右端边界对最后一行作同样处理。本题两个边界值均为零，但程序仍保留一般边界修正，使求解函数可接收任意给定的两端函数值。本文实验采用题目规定的零边界条件。

2.2 稀疏线性系统的 MATLAB 实现

程序 2-1 网格生成、稀疏组装与边界处理

```
h = (xSpan(2) - xSpan(1)) / nIntervals;
x = linspace(xSpan(1), xSpan(2), nIntervals + 1).';
xInner = x(2:end-1);
nInner = nIntervals - 1;
mainDiagonal = -2 * ones(nInner, 1);
offDiagonal = ones(nInner, 1);
A = spdiags([offDiagonal, mainDiagonal, offDiagonal], [-1, 0, 1], nInner, nInner) / h^2;
rhs = sourceFun(xInner);
rhs = rhs(:);
rhs(1) = rhs(1) - boundaryValues(1) / h^2;
rhs(end) = rhs(end) - boundaryValues(2) / h^2;
uInner = A \ rhs;
uNum = [boundaryValues(1); uInner; boundaryValues(2)];
```

`linspace` 生成包括两端点的 $N + 1$ 个坐标，末尾的转置将其变为列向量；`x(2:end-1)` 选取内部节点。`spdiags` 将三个列向量放到编号为 -1 、 0 、 1 的对角线上，只存储三对角结构中的非零元素。内部矩阵规模为 $(N - 1) \times (N - 1)$ ，这样可以直接生成稀疏三对角矩阵，减少存储和组装开销。

`rhs(:)` 将源项输出整理为列向量，使线性系统两端尺寸一致。内部解由 MATLAB 反斜杠直接求得，再与左右边界值拼接成完整节点向量。`meta` 保存矩阵 A 、右端项、线性残差和条件数估计，便于检查离散系统的求解情况；数值精度则通过与解析解比较另行评价。

2.3 数值解与空间误差分布

图 2-1 选取代表性网格比较数值解与解析解。曲线具有偶对称性，内部为负，端点为零，与解析性质一致。随着网格加密，两条曲线在普通纵轴下逐渐重合，具体精度由误差曲线和数值指标进一步说明。

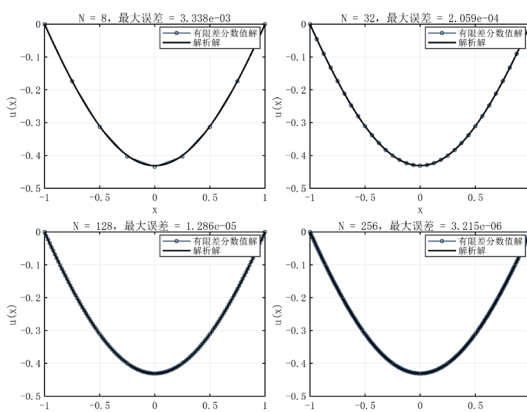


图 2-1 有限差分数值解与解析解的代表网格比较

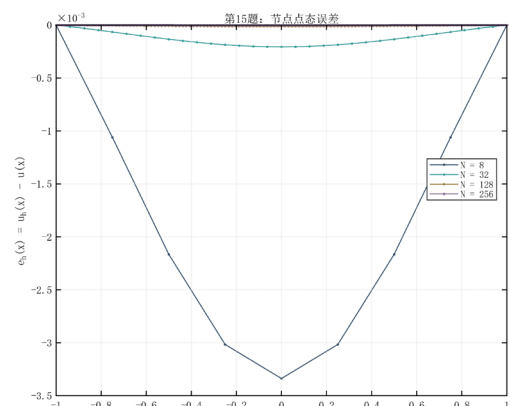


图 2-2 有限差分节点有符号误差

图 2-2 绘制数值解减去解析解的有符号误差。误差在边界处为零，在区间内部平滑变化；网格加密后，误差幅值明显减小，而空间分布形态基本保持一致。这与中心差分截断误差的连续分布相符，也便于发现节点错位等实现问题。

2.4 二阶收敛与残差分析

表 2-1 有限差分正式收敛结果

N	h	最大误差	RMS 误差	相邻阶
8	0.25	3.3376e-03	2.1334e-03	—
16	0.125	8.2587e-04	5.4357e-04	2.0148
32	0.0625	2.0594e-04	1.3761e-04	2.0037
64	0.03125	5.1453e-05	3.4645e-05	2.0009
128	0.015625	1.2861e-05	8.6935e-06	2.0002
256	0.0078125	3.2152e-06	2.1775e-06	2.0001
512	0.00390625	8.0379e-07	5.4491e-07	2.0000

表 2-1 中最大误差从 3.3376×10^{-3} 降低到 8.0379×10^{-7} ，相邻观测阶逐渐趋近 2，七级网格的拟合阶为 2.002502。网格加密一倍后误差约缩小为原来的四分之一，与中心差分的二阶行为一致。

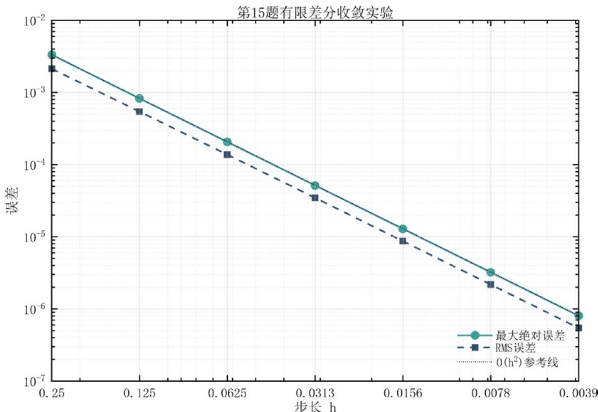


图 2-3 有限差分正式误差收敛

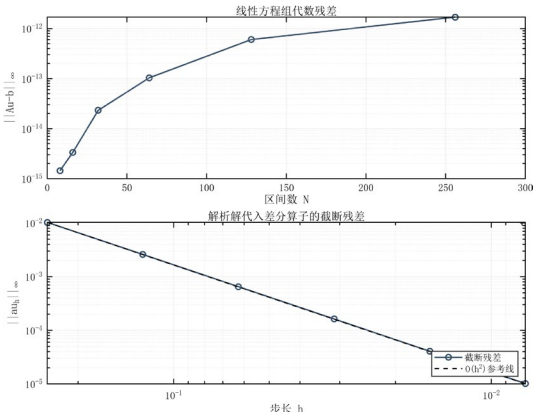


图 2-4 代数残差与局部截断残差

代数残差和截断残差反映不同环节。代数残差由数值解代回线性系统得到，用于检查离散方程的求解精度；截断残差由解析解节点值代入差分算子得到，用于衡量离散算子对连续方程的近似误差。两者分别为

$$r_{\text{alg}} = AU - b, \tau_i = \frac{u(x_{i-1}) - 2u(x_i) + u(x_{i+1}))}{h^2} - e^{-x_i^2} \quad (2.5)$$

基础六级网格中，代数残差最大范数由约 1.44×10^{-15} 变化到 1.70×10^{-12} ，截断残差则由 1.0288×10^{-2} 减小到 1.0172×10^{-5} ，并呈二阶趋势。代数残差很小，说明离散方程已被充分求解；真实误差仍主要来自离散近似。图 2-4 分别展示两类残差及其量级。

三、Richardson 外推与精度提升

3.1 主误差项消去

对于光滑解和均匀嵌套网格，中心差分解在共同节点上的误差可展开为步长的偶次幂^[2]。Richardson 外推利用已知的主误差阶组合粗、细网格结果，从而消去二阶主误差项^[1]。

$$U_h = u + c_2 h^2 + c_4 h^4 + O(h^6), U_{\frac{h}{2}} = u + \frac{c_2 h^2}{4} + \frac{c_4 h^4}{16} + O(h^6) \quad (3.1)$$

$$U_R = \frac{4U_{\frac{h}{2}} - U_h}{3} = u - \frac{c_4 h^4}{4} + O(h^6) \quad (3.2)$$

式(3.2)消去二阶项后，结果的预期阶数提高到四阶。系数 4 来自步长减半时二阶误差缩小为原来的四分之一。该组合要求原方法以二阶误差为主，并且粗、细网格数据位于同一节点。由于还需计算一次细网格解，精度提升伴随额外计算量。

3.2 粗细网格对应与 MATLAB 实现

程序 3-1 两级网格求解与 Richardson 组合

```
[x, uCoarse, coarseMeta] = solvePoissonFD( ...
    nIntervals, sourceFun, boundaryValues, xSpan);
[~, uFine, fineMeta] = solvePoissonFD( ...
    2 * nIntervals, sourceFun, boundaryValues, xSpan);

% 细网格的第 1、3、5、... 个元素与粗网格节点位于相同位置。
uFineAtCoarse = uFine(1:2:end);

uRichardson = (4 * uFineAtCoarse - uCoarse) / 3;
```

代码来源: src/poisson/richardsonExtrapolatePoisson.m (核心片段, 完整函数见附录或配套源程序)。

粗网格有 $N + 1$ 个节点，细网格有 $2N + 1$ 个节点。MATLAB 从 1 开始索引，因此细网格第 1、3、5 等奇数位置恰好对应粗网格节点，`uFine(1:2:end)` 得到长度为 $N + 1$ 的列向量。粗、细网格节点在这些位置直接对应，因此可以在共同节点上完成外推组合。

外推函数在相同源项、边界和区间上调用 `solvePoissonFD` 两次，并保存粗网格解、细网格共同节点值、外推解和解析解。由此既能比较最大误差，也能逐节点观察外推前后的变化。程序中的组合式虽然简短，其正确使用仍取决于误差阶和节点对应关系。

3.3 精度提升与观测阶

表 3-1 Richardson 外推前后的最大误差

粗网格 N	细网格 2N	差分误差	外推误差	外推相邻阶
8	16	3.3376e-03	1.1382e-05	—
16	32	8.2587e-04	6.9958e-07	4.0241

粗网格 N	细网格 2N	差分误差	外推误差	外推相邻阶
32	64	2.0594e-04	4.3543e-08	4.0060
64	128	5.1453e-05	2.7187e-09	4.0015
128	256	1.2861e-05	1.6986e-10	4.0005
256	512	3.2152e-06	1.0586e-11	4.0041

表 3-1 显示，粗网格 $N = 16$ 时，普通有限差分最大误差为 8.2587×10^{-4} ，而 Richardson 外推误差为 6.9958×10^{-7} ；两者均在粗网格节点上与同一解析解比较。随粗网格继续加密，外推的相邻观测阶保持在 4 附近，最后一组最大误差降至 1.0586×10^{-11} 。误差比的提高与阶数的变化共同支持主误差项被消去的解释，而不只是单一网格上的偶然误差变小。

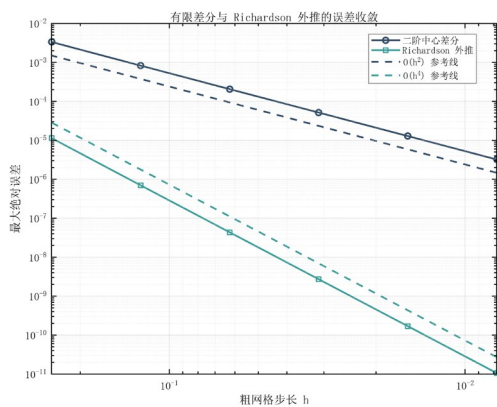


图 3-1 有限差分与 Richardson 外推的收敛比较

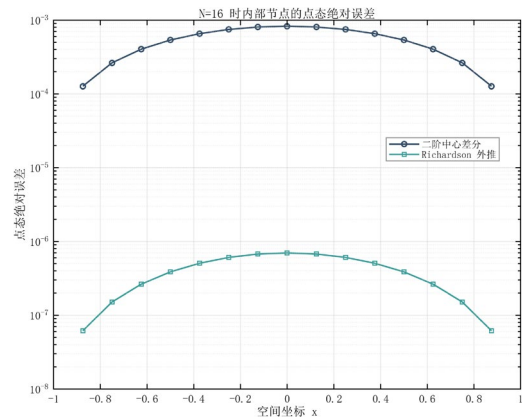


图 3-2 $N=16$ 时内部共同节点的外推误差比较

图 3-2 展示 $N=16$ 时误差在区间内的分布。外推后，各内部节点的误差均明显减小，精度改善覆盖整个区间。由于边界误差严格为零，对数坐标图仅绘制内部节点。上述结果对应本文的光滑算例和六组嵌套网格。

四、常微分方程初值问题的 RK4 求解

4.1 解析解推导与符号计算

将初值问题写为 $y' + y = t^2 + 3$ ，乘以积分因子 e^t 后，左端成为乘积 $e^t y$ 的导数。积分可得多项式特解 $t^2 - 2t + 5$ 与指数齐次项之和，再由 $y(0) = 1$ 确定系数为 -4 ，因此

$$y(t) = t^2 - 2t + 5 - 4e^{-t} \quad (4.1)$$

在 MATLAB 中，`ivpExactSymbolic` 先用 `dsolve` 求解带初值的符号方程，再通过 `matlabFunction` 生成可作用于节点向量的函数。程序用 `simplify` 分别计算符号解与手推式之差、方程残差和初值残差。`ode_method_comparison.mat` 中的 `exactCheck` 显示三项残差均为零，`passed` 为真。

程序 4-1 符号解析解与判零检查

```

syms t y(t)
equation = diff(y, t) == -y + t^2 + 3;
ySym = simplify(dsolve(equation, y(0) == 1));
handDerived = t^2 - 2*t + 5 - 4*exp(-t);
verification.handDerived = handDerived;
verification.differenceResidual = simplify(ySym - handDerived);
verification.equationResidual = simplify(diff(ySym, t) + ySym - t^2 - 3);
verification.initialValue = simplify(subs(ySym, t, 0));
verification.initialResidual = simplify(verification.initialValue - 1);

```

代码来源: src/ode/ivpExactSymbolic.m (核心片段, 完整函数见附录或配套源程序)。

符号计算和数值迭代彼此独立。解析函数只用于计算完成后的误差评价, RK4 的四个阶段斜率均由微分方程右端给出。手推表达式与 Symbolic Math Toolbox 生成的符号解分别保留, 便于核对两种推导结果。

4.2 经典 RK4 与向量状态实现

设右端函数为 $f(t, y)$ 。RK4 在一个步长内计算起点、两个中点预测状态和终点预测状态的斜率, 再按 1、2、2、1 加权更新。本文的 k 表示斜率, 不含步长, 因而更新公式外还要乘 h 。若把 h 预先乘入 k , 可得到教材中另一种常见记法。两种写法等价, 本文统一采用前者^[1]。

$$Y_{n+1} = Y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (4.2)$$

对于足够光滑的右端函数, 经典 RK4 在精确初值出发的单步误差为 $O(h^5)$, 在有限区间的通常收敛条件下累计全局误差为 $O(h^4)$ ^[3]。本文关注有限区间上的全局误差, 并通过整个节点序列的最大误差检验四阶收敛。

程序 4-2 RK4 核心循环与四个阶段斜率

```

for iStep = 1:nSteps
    time = t(iStep);
    state = yNum(iStep, :).';
    k1 = checkedOdeSlope(odeFun, time, state);
    k2 = checkedOdeSlope(odeFun, time + h/2, state + h*k1/2);
    k3 = checkedOdeSlope(odeFun, time + h/2, state + h*k2/2);
    k4 = checkedOdeSlope(odeFun, t(iStep + 1), state + h*k3);
    nextState = state + h*(k1 + 2 * k2 + 2 * k3 + k4) / 6;

```

代码来源: src/ode/rk4Solve.m (核心片段, 完整函数见附录或配套源程序)。

prepareOdeGrid 预先建立时间列向量及 $(N + 1) \times d$ 的解数组, d 为初始状态的分量数。每行保存一个时间节点、每列保存一个状态分量; 计算斜率时通过非共轭转置把当前行转为列状态, 更新结束后再转回行写入。对当前标量问题 $d=1$, 对后续打靶法 $d=2$, 同一更新式仍然适用。

数组预分配可减少循环中的动态扩展。checkedOdeSlope 将右端函数输出整理为 double 列向量, 主循环同时检查新状态中的 NaN 和 Inf。函数接口要求时间区间递增、

步数为正整数，并保证状态维度一致。RK4 每步计算四次右端函数，meta 记录的总调用次数为 $4N$ 。

4.3 数值解与有符号误差

图 4-1 比较最粗、最细基础网格上的 RK4 解与解析解，两组数值解都能反映解的主要变化。为观察普通坐标下不易分辨的偏差，图 4-2 进一步绘制 $Y-y$ 的有符号误差。初值由程序直接赋给数值解，后续误差来自逐步离散及其传播。

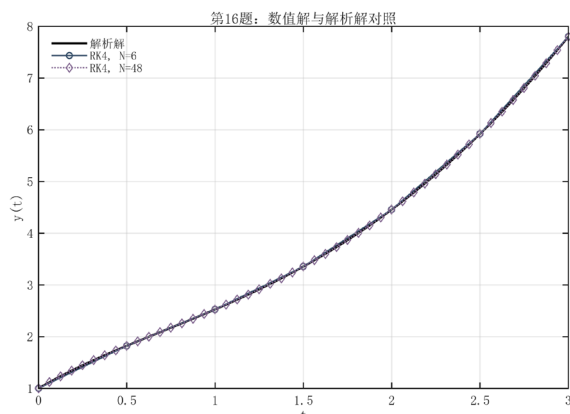


图 4-1 RK4 数值解与解析解比较

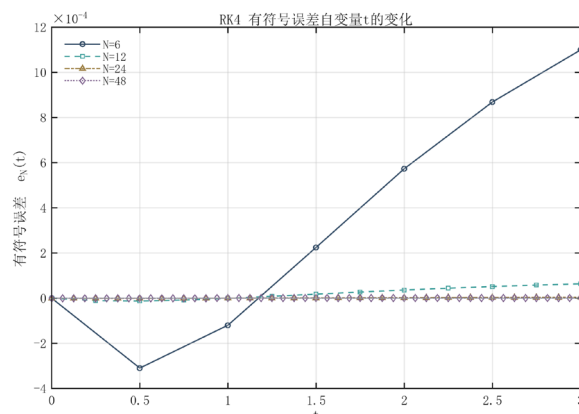


图 4-2 RK4 有符号误差随自变量的变化

有符号误差可以区分数值解偏高或偏低，并显示偏差较集中的区间。不同步长的误差幅值相差较大，小步长曲线因纵轴尺度而靠近零线。结合表 4-1 中的最大误差，可以更准确地判断各组结果的精度。

图 4-2 中，最粗网格的误差先为负，随后转正，并在区间后部增大，说明误差方向会随时间变化。即使初值精确，步进过程仍会产生并传播离散误差。最大绝对误差覆盖全部节点，能够补充终点误差所遗漏的信息。

4.4 四阶收敛与归一化误差

表 4-1 RK4 正式收敛结果

N	h	最大误差	RMS 误差	相邻阶
6	0.5	1.0994e-03	5.9197e-04	—
12	0.25	6.3431e-05	3.3685e-05	4.1154
24	0.125	3.7922e-06	2.0055e-06	4.0641
48	0.0625	2.3157e-07	1.2227e-07	4.0335
96	0.03125	1.4302e-08	7.5470e-09	4.0171
192	0.015625	8.8854e-10	4.6872e-10	4.0087
384	0.0078125	5.5366e-11	2.9202e-11	4.0044

表 4-1 中，最大误差由 1.0994×10^{-3} 降至 5.5366×10^{-11} ，七级网格的拟合阶为 4.036671。较粗两级的相邻阶为 4.1154，随后逐渐趋近 4，表明粗步长下仍含有较明显的高阶误差项。相邻阶的变化和整体拟合共同体现了 RK4 的四阶收敛特征。

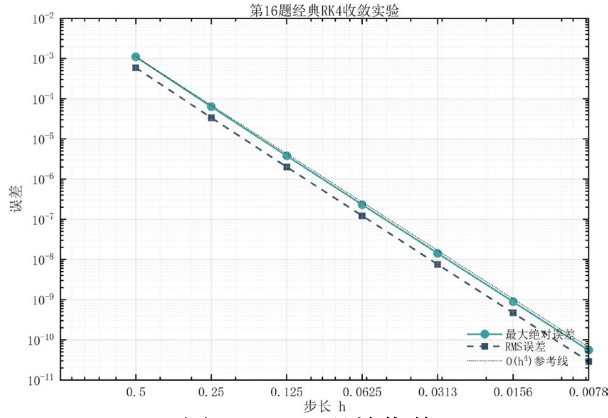


图 4-3 RK4 误差收敛

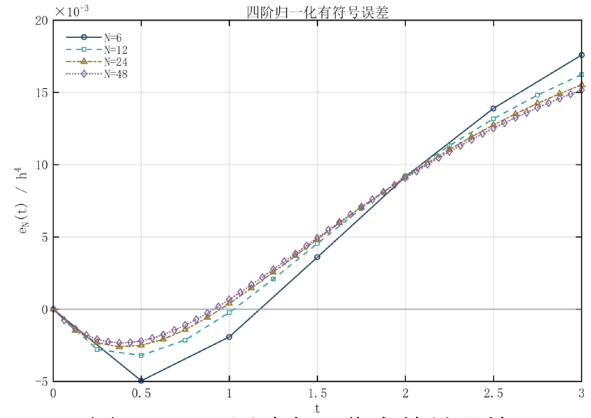


图 4-4 RK4 四阶归一化有符号误差

当全局误差写成 $h^4 C(t)$ 与更高阶项之和时，有符号误差除以 h^4 后应逐渐接近同一函数 $C(t)$ 。图 4-4 用四组基础网格观察这一特征。归一化曲线着重显示误差形状和主项系数，与图 4-3 的整体收敛趋势相互补充。

图 4-4 中的归一化曲线均呈先负后正的形状，且细网格曲线更加接近，表明它们具有相近的主误差函数。最粗网格在区间末端仍有可见偏差，与表 4-1 中粗步长相邻阶偏离 4 的现象一致。连续加密网格后，四阶渐近特征逐渐清晰。

五、Euler、RK2 与 RK4 的精度和效率比较

5.1 更新公式与统一比较接口

显式 Euler 法用起点斜率直接推进。本文的 RK2 先用起点斜率预测终点状态，再取起点与预测终点斜率的平均值完成更新，即 Heun 型显式梯形格式。下式给出本文所采用的具体算法。

$$Y_{n+1} = Y_n + hf(t_n, Y_n) \quad (5.1)$$

$$\begin{aligned} k_1 &= f(t_n, Y_n) \\ k_2 &= f(t_n + h, Y_n + k_1 h) \\ Y_{n+1} &= Y_n + \frac{k_1 + k_2}{2} h \end{aligned} \quad (5.2)$$

三个求解器采用相同接口，均输入右端函数、时间区间、初值和步数，返回时间节点、解矩阵与 meta。比较程序在同一组参数上依次调用各方法，并统一使用 errorNorms 和 computeOrder 计算误差与阶数。Euler、RK2 和 RK4 每步的右端函数调用数分别为 1、2 和 4。

程序 5-1 RK2 的预测与校正更新

```
for iStep = 1:nSteps
    state = yNum(iStep, :).';
    k1 = checkedOdeSlope(odeFun, t(iStep), state);
    k2 = checkedOdeSlope(odeFun, t(iStep + 1), state + h*k1);
    nextState = state + h*(k1 + k2)/2;
```

代码来源：src/ode/rk2Solve.m（核心片段，完整函数见附录或配套源程序）。

5.2 相同步长下的误差与阶数

表 5-1 三方法最大误差与拟合阶

方法	N=6	N=48	N=384	拟合阶
Euler	3.5554e-01	4.1115e-02	5.0959e-03	1.0181
RK2	1.1746e-01	1.5075e-03	2.2999e-05	2.0470
RK4	1.0994e-03	2.3157e-07	5.5366e-11	4.0367

表 5-1 和图 5-1 表明，在相同步长下，本算例的误差由小到大依次为 RK4、RK2 和 Euler。三种方法的拟合阶分别为 4.036671、2.046956 和 1.018057，与相应的四阶、二阶和一阶理论一致。网格加密时，各方法的误差都在下降，但下降速度明显不同。

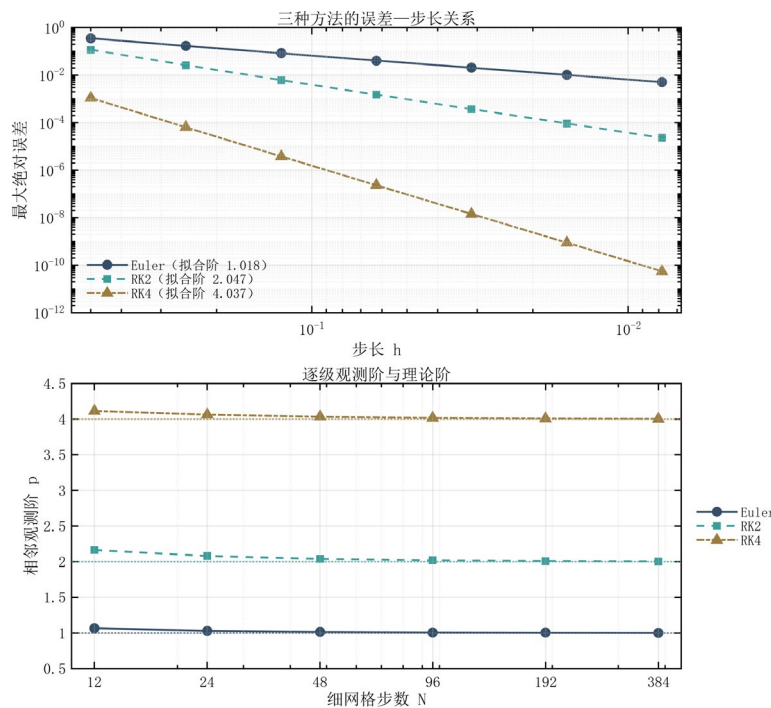


图 5-1 三种单步方法的误差收敛及观测阶

5.3 函数调用次数与精度

相同步长对应的计算量并不相同，因此进一步以右端函数调用次数 W 衡量主要工作量。对固定方法有 $W = qN = qL/h$ ，其中 q 为每步阶段数， L 为积分区间长度。当误差近似为 Ch^p 时，误差随 W 约按 W^{-p} 下降。高阶方法虽然每步需要更多阶段，但在较高精度要求下可能用较少的总调用数达到目标；实际效果还取决于误差常数。

表 5-2 相同右端调用次数 $W=96$ 的实际比较

方法	步数 N	调用次数 W	最大误差
Euler	96	96	2.0457e-02
RK2	48	96	1.5075e-03
RK4	24	96	3.7922e-06

表 5-2 选取右端函数调用次数同为 96 的三组结果。Euler、RK2 和 RK4 的最大误差分别为 2.0457×10^{-2} 、 1.5075×10^{-3} 和 3.7922×10^{-6} ；在这一计算预算下，RK4 的精度最高。函数调用次数便于复核，也较少受机器状态影响，但未包含数组操作和函数管理等开销，因此还需结合实测时间分析。

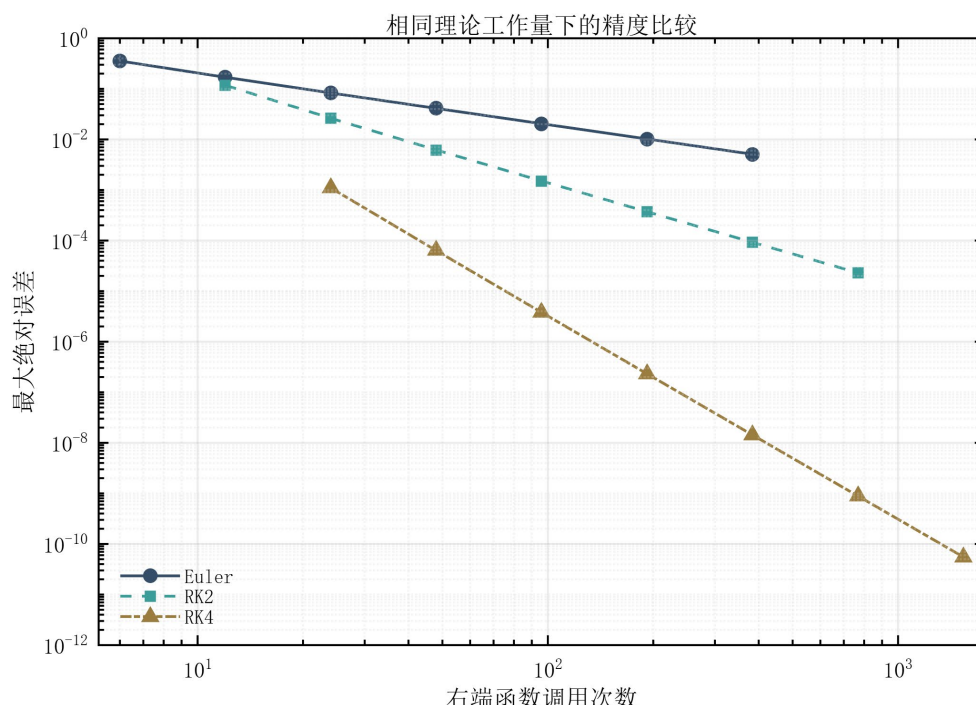


图 5-2 三方法的函数调用次数与精度关系

5.4 实测时间与结论边界

计时前先对每个方法与步数组组合预热，随后执行七轮 `timeit`，并轮换方法的运行顺序。计时范围仅包含求解器调用，符号求解、误差计算、绘图和文件写入均在计时之外。每组保存均值、中位数、标准差、最小值和最大值，正文以中位数为代表。

表 5-3 N=48 时的本机计时与精度

方法	中位耗时/ μ s	时间变异系数/%	最大误差
Euler	3.508	0.880	4.1115e-02
RK2	4.521	0.765	1.5075e-03
RK4	6.565	0.504	2.3157e-07

表 5-3 显示，在 48 步条件下，方法阶数越高，单次求解时间有所增加，但误差下降更为显著。因此，同步数下的运行速度与达到给定精度所需的时间应分别讨论。图 5-3 同时给出精度和实测时间；全部组合的最大时间变异系数约为 3.11%，详细波动见附录 B。

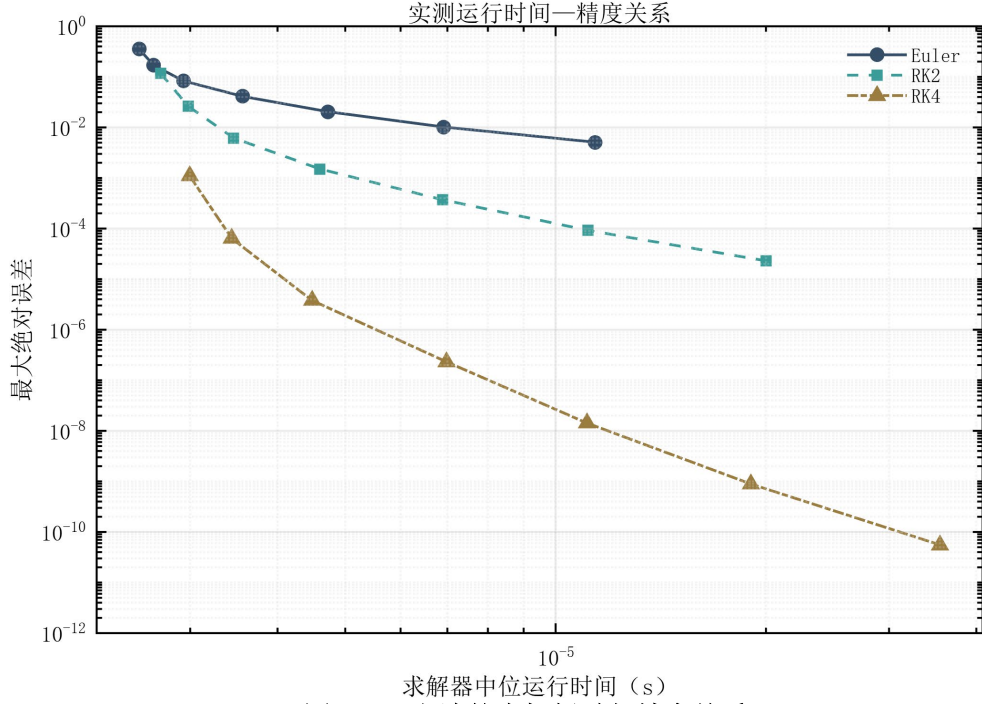


图 5-3 三方法的本机耗时与精度关系

本算例的右端函数较简单，单次求解只有微秒量级，固定调用开销和系统状态都会影响计时。本文的时间结果用于描述当前软硬件环境下的运行情况，方法效率主要结合误差、函数调用次数和计时三项指标进行比较。

六、基于 RK4 的线性打靶法与交叉验证

6.1 边值问题向初值问题的转化

有限差分一次求得所有内部节点值，而打靶法把未知的左端斜率作为参数，通过初值积分使右端值满足边界条件。令 $v = u'$ ，Poisson 方程可改写为由两个一阶方程组成的系统，状态向量为 $[u, v]^T$ 。线性打靶的一般方法见文献^[1]；本题源项仅依赖 x ，因此修正过程可以简化。

$$\frac{d}{dx} \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} v \\ e^{-x^2} \end{pmatrix} \quad (6.1)$$

设 z 满足左端函数值为 α 且左端斜率为零。初始斜率增加 s 时，对应的齐次解为 $s(x - a)$ 。因此，先用 RK4 计算一次 z ，再根据右端边界 β 即可确定斜率修正量。本文取 $a = -1$ 、 $b = 1$ 、 $\alpha = \beta = 0$ ，是该公式的一个特例。

$$s = \frac{\beta - z(b)}{b - a} \quad (6.2)$$

$$u_s(x) = z(x) + s(x - a)$$

由于齐次解已知为线性函数，本题可用一次 RK4 积分和一次线性修正确定初始斜率，省去了牛顿法或割线法的重复搜索。这一简化来自当前方程的线性结构；当右端显式依赖 u 或 u' 时，修正关系需要重新推导。

6.2 MATLAB 复用与端点处理

程序 6-1 向量 RK4 调用与线性斜率修正

```
odeFun = @(xValue, state) [state(2); checkedSource(sourceFun, xValue)];
[x, stateZeroSlope, rkMeta] = rk4Solve(odeFun, xSpan, [leftBoundary; 0], nIntervals);

intervalLength = xSpan(2) - xSpan(1);
initialSlope = (rightBoundary - stateZeroSlope(end, 1)) / intervalLength;
uNum = stateZeroSlope(:, 1) + initialSlope .* (x - xSpan(1));
uNum(1) = leftBoundary;
uNum(end) = rightBoundary;
代码来源: src/poisson/solvePoissonShooting.m (核心片段, 完整函数见附录或配套源程序)。
```

匿名函数输入位置和两分量列状态，返回 $[state(2), f(x)]$ 的转置。rk4Solve 输出矩阵的第一列为零斜率解 z ，其终点值即 $z(b)$ 。求得 $initialSlope$ 后，程序在全部节点上加入相应线性项，并显式写回两端边界值。这一实现体现了向量 RK4 接口在二阶方程组中的复用。

显式写回边界可消除端点浮点运算产生的微小偏差。内部精度则通过解析解误差和网格收敛检验。打靶法每步进行四次向量右端调用，总调用次数为 $4N$ ；由于状态维数不同，这一工作量指标主要用于同一打靶实验内部的复核。

6.3 三方比较与误差解释

表 6-1 打靶法、有限差分及两者差值的最大范数

N	打靶误差	有限差分误差	两方法差值	相邻阶
8	1.1382e-05	3.3376e-03	3.3490e-03	—
16	6.9958e-07	8.2587e-04	8.2657e-04	4.0241
32	4.3543e-08	2.0594e-04	2.0599e-04	4.0060
64	2.7187e-09	5.1453e-05	5.1456e-05	4.0015
128	1.6987e-10	1.2861e-05	1.2861e-05	4.0004
256	1.0616e-11	3.2152e-06	3.2152e-06	4.0001
512	6.6303e-13	8.0379e-07	8.0379e-07	4.0010

表 6-1 显示，打靶法、有限差分及两方法差值的拟合阶分别为 4.004176、2.002502 和 2.003125。打靶法最大误差由 1.1382×10^{-5} 降至 6.6303×10^{-13} ，在相同网格下明显小于有限差分误差。图 6-1 比较代表网格上的解和点态绝对误差，图 6-2 给出全部网格的收敛趋势。

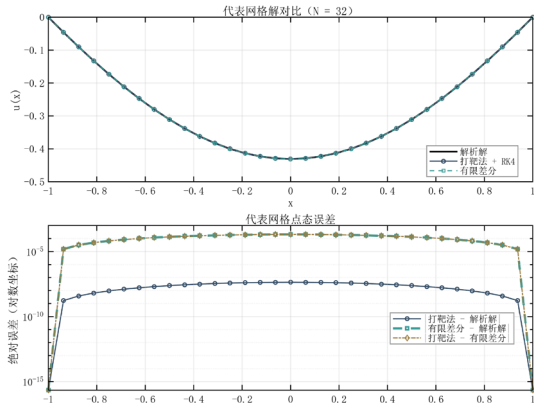


图 6-1 打靶法、有限差分和解析解的比较

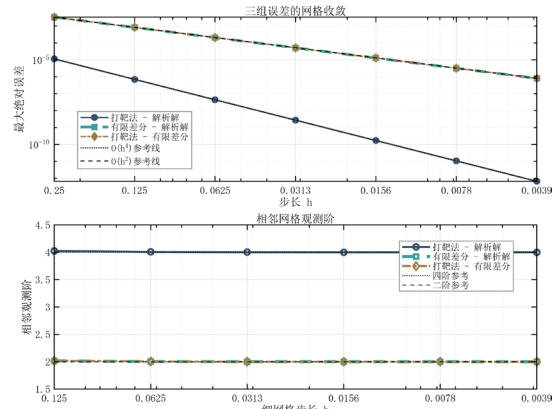


图 6-2 打靶法误差及方法间差值的收敛

设打靶解和有限差分解分别为 $u + eS$ 与 $u + eFD$ ，则两者之差为 $eS - eFD$ 。打靶误差为四阶，有限差分误差为二阶，因此在较细网格上，方法间差值由二阶的 eFD 主导。这解释了两方法差值呈二阶，而打靶法相对解析解仍保持四阶。

图 6-1 采用对数误差坐标，绘图时将零值替换为机器精度下限，原始 CSV 和 MAT 数据保持不变。打靶法与有限差分从不同计算路径求解同一问题，并共同与解析解比较，为程序和阶数检验提供了交叉参照。相关结论对应本文给定的光滑源项、区间和边界条件。

七、结论

本文完成了一维 Poisson 边值问题和常微分方程初值问题的 MATLAB 数值求解，并将公式推导、程序实现和误差分析贯穿于实验过程。有限差分利用稀疏三对角系统处理边界约束，RK4 通过四阶段斜率逐步推进初值。两题均以解析解为参照，并保存节点数据、汇总指标和图形结果，便于复核计算过程。

七级网格实验中，有限差分和 RK4 最大误差的拟合阶分别为 2.002502 和 4.036671，与理论二阶、四阶一致。代数残差表明线性系统已被准确求解，截断残差则揭示了离散近似的误差来源。Richardson 外推组合共同节点上的粗、细网格解，消去二阶主误差项后，相邻观测阶提高到约 4，显著改善了有限差分精度。

Euler、RK2 和 RK4 的拟合阶依次接近 1、2 和 4。在相同 96 次右端函数调用下，RK4 取得最小误差，体现了高阶方法在本算例中的精度优势。实测时间还受到阶段数、程序开销和机器状态影响，因此效率评价同时采用函数调用次数和本机计时结果。

线性打靶法调用向量 RK4 并进行一次斜率修正，所得拟合阶为 4.004176，由此建立了边值求解与初值推进之间的联系。程序实现中的主要难点包括边界项符号、MATLAB 行列向量约定、粗细网格索引对应和误差指标区分。统一的函数接口及中间诊断量使这些问题能够在计算过程中被定位和检查。

本项目以经典数值方法的完整实现和误差验证为重点。两个算例均较光滑且具有解析解，计时数据也来自单一软硬件环境；更复杂的源项、非零边界和非线性问题还需另行设计验证实验。通过本次实践，数学推导、MATLAB 函数、节点结果和误差结论形成了清晰对应，程序复用的接口条件与数学依据也得到具体体现。

参考文献

- [1] Burden, R. L., Faires, J. D., & Burden, A. M. (2016). Numerical analysis (10th ed.). Cengage Learning.
- [2] 李庆扬, 王能超, 易大义. (2008). 数值分析 (第 5 版). 清华大学出版社.
- [3] LeVeque, R. J. (2007). Finite difference methods for ordinary and partial differential equations: Steady-state and time-dependent problems. Society for Industrial and Applied Mathematics.

附录 A 核心算法源代码

以下代码提取自配套源文件，保留实际运算语句，并省略空行和整行注释。排版中部分长行会自动换行，实际运行应使用配套.m 文件。正文已说明主要变量和算法关系，本附录列出完整求解函数，实验与绘图程序见附录 C 索引。

A.1 solvePoissonFD.m

源文件: src/poisson/solvePoissonFD.m。

```
function [x, uNum, meta] = solvePoissonFD(nIntervals, sourceFun, boundaryValues, xSpan)
    if nargin < 1 || isempty(nIntervals), nIntervals = 32; end
    if nargin < 2 || isempty(sourceFun), sourceFun = @(x) exp(-x.^2); end
    if nargin < 3 || isempty(boundaryValues), boundaryValues = [0, 0]; end
    if nargin < 4 || isempty(xSpan), xSpan = [-1, 1]; end
    validateattributes(nIntervals, {'numeric'}, {'scalar', 'integer', '>='}, 2);
    validateattributes(boundaryValues, {'numeric'}, {'vector', 'numel', 2, 'finite'});
    validateattributes(xSpan, {'numeric'}, {'vector', 'numel', 2, 'finite'});
    h = (xSpan(2) - xSpan(1)) / nIntervals;
    x = linspace(xSpan(1), xSpan(2), nIntervals + 1).';
    xInner = x(2:end-1);
    nInner = nIntervals - 1;
    mainDiagonal = -2 * ones(nInner, 1);
    offDiagonal = ones(nInner, 1);
    A = spdiags([offDiagonal, mainDiagonal, offDiagonal], [-1, 0, 1], nInner, nInner) /
    h^2;
    rhs = sourceFun(xInner);
    rhs = rhs(:);
    rhs(1) = rhs(1) - boundaryValues(1) / h^2;
    rhs(end) = rhs(end) - boundaryValues(2) / h^2;
    uInner = A \ rhs;
    uNum = [boundaryValues(1); uInner; boundaryValues(2)]; % 由有限差分法所得的解
    meta.stepSize = h; % 子区间跨度
    meta.nIntervals = nIntervals; % 子区间数量
    meta.matrix = A; % 稀疏三对角线矩阵
    meta.rhs = rhs; % 右端项
    meta.sourceValues = sourceFun(xInner); % 区间内点处函数值
    meta.matrixSize = size(A);
    meta.linearResidual = A * uInner - rhs; % 残差
    meta.conditionEstimate = condest(A);
end
```

A.2 poissonExact.m

源文件: src/poisson/poissonExact.m。

```
function uExact = poissonExact(x)
    validateattributes(x, {'numeric'}, {'real', 'finite'});
    uExact = 0.5 .* exp(-x.^2) + 0.5 .* sqrt(pi) .* x .* erf(x) - 0.5 .* exp(-1) - 0.5
    .* sqrt(pi) .* erf(1);
end
```

A.3 richardsonExtrapolatePoisson.m

源文件: src/poisson/richardsonExtrapolatePoisson.m。

```
function [x, uRichardson, meta] = richardsonExtrapolatePoisson(nIntervals, sourceFun, boundaryValues, xSpan)
    if nargin < 1 || isempty(nIntervals), nIntervals = 32; end
    if nargin < 2 || isempty(sourceFun), sourceFun = @(x) exp(-x.^2); end
    if nargin < 3 || isempty(boundaryValues), boundaryValues = [0, 0]; end
    if nargin < 4 || isempty(xSpan), xSpan = [-1, 1]; end
    validateattributes(nIntervals, {'numeric'}, {'scalar', 'integer', '>='}, 2);
    [x, uCoarse, coarseMeta] = solvePoissonFD( ...
        nIntervals, sourceFun, boundaryValues, xSpan);
    [~, uFine, fineMeta] = solvePoissonFD( ...
        2 * nIntervals, sourceFun, boundaryValues, xSpan);
    uFineAtCoarse = uFine(1:2:end);
    uRichardson = (4 * uFineAtCoarse - uCoarse) / 3;
    meta.coarseN = nIntervals;
    meta.fineN = 2 * nIntervals;
    meta.stepSize = coarseMeta.stepSize;
    meta.coarseSolution = uCoarse;
    meta.fineSolutionAtCoarseNodes = uFineAtCoarse;
    meta.coarseSolver = coarseMeta;
    meta.fineSolver = fineMeta;
end
```

A.4 ivpExactSymbolic.m

源文件: src/ode/ivpExactSymbolic.m。

```
function [ySym, yExact, verification] = ivpExactSymbolic()
    syms t y(t)
    equation = diff(y, t) == -y + t^2 + 3;
    ySym = simplify(dsolve(equation, y(0) == 1));
    handDerived = t^2 - 2*t + 5 - 4*exp(-t);
    verification.handDerived = handDerived;
    verification.differenceResidual = simplify(ySym - handDerived);
    verification.equationResidual = simplify(diff(ySym, t) + ySym - t^2 - 3);
    verification.initialValue = simplify(subs(ySym, t, 0));
    verification.initialResidual = simplify(verification.initialValue - 1);
    residuals = [verification.differenceResidual, ...
        verification.equationResidual, verification.initialResidual];
    verification.passed = all(isAlways(residuals == 0, 'Unknown', 'false'));
    yExact = matlabFunction(ySym, 'Vars', t);
    verification.explicitFunction = @(time) time.^2 - 2*time + 5 - 4*exp(-time);
end
```

A.5 rk4Solve.m

源文件: src/ode/rk4Solve.m。

```
function [t, yNum, meta] = rk4Solve(odeFun, tSpan, y0, nSteps)
    [t, yNum, h] = prepareOdeGrid(tSpan, y0, nSteps);
    for iStep = 1:nSteps
        time = t(iStep);
        state = yNum(iStep, :).';
        k1 = checkedOdeSlope(odeFun, time, state);
        k2 = checkedOdeSlope(odeFun, time + h/2, state + h*k1/2);
        k3 = checkedOdeSlope(odeFun, time + h/2, state + h*k2/2);
        k4 = checkedOdeSlope(odeFun, t(iStep + 1), state + h*k3);
        nextState = state + h*(k1 + 2 * k2 + 2 * k3 + k4) / 6;
        if any(~isfinite(nextState))
            break;
        end
        yNum(iStep + 1, :) = nextState';
    end
end
```

```

        error('rk4Solve:NonfiniteState', '第 %d 步的数值解出现 NaN/Inf。', iStep);
    end
    yNum(iStep + 1, :) = nextState.';
end
meta = struct('stepSize', h, 'nSteps', nSteps, 'method', '经典固定步长 RK4',
'functionEvaluations', 4*nSteps);
end
function [t, yNum, h] = prepareOdeGrid(tSpan, y0, nSteps)
    tSpan = double(tSpan); nSteps = double(nSteps);
    h = (tSpan(2) - tSpan(1)) / nSteps;
    t = linspace(tSpan(1), tSpan(2), nSteps + 1).';
    yNum = zeros(nSteps + 1, numel(y0)); yNum(1,:) = double(y0(:)).';
end
function slope = checkedOdeSlope(odeFun,time,state)
    slope = odeFun(time,state);
    slope = double(slope(:));
end

```

A.6 eulerSolve.m

源文件: src/ode/eulerSolve.m。

```

function [t, yNum, meta] = eulerSolve(odeFun, tSpan, y0, nSteps)
    [t, yNum, h] = prepareOdeGrid(odeFun, tSpan, y0, nSteps);
    for iStep = 1:nSteps
        state = yNum(iStep, :).';
        slope = checkedOdeSlope(odeFun, t(iStep), state);
        nextState = state + h*slope;
        if any(~isfinite(nextState))
            error('eulerSolve:NonfiniteState', '第 %d 步的数值解出现 NaN/Inf。', iStep);
        end
        yNum(iStep + 1, :) = nextState.';
    end
    meta = struct('stepSize', h, 'nSteps', nSteps, ...
        'method', '显式 Euler', 'functionEvaluations', nSteps);
end
function [t, yNum, h] = prepareOdeGrid(~,tSpan,y0,nSteps)
    nSteps=double(nSteps); h=(tSpan(2)-tSpan(1))/nSteps;
    t=linspace(tSpan(1),tSpan(2),nSteps+1).'; yNum=zeros(nSteps+1,numel(y0));
    yNum(1,:)=double(y0(:)).';
end
function slope=checkedOdeSlope(f,time,state)
    slope = f(time,state);
    slope=double(slope(:));
end

```

A.7 rk2Solve.m

源文件: src/ode/rk2Solve.m。

```
function [t, yNum, meta] = rk2Solve(odeFun, tSpan, y0, nSteps)
    [t, yNum, h] = prepareOdeGrid(odeFun, tSpan, y0, nSteps);
    for iStep = 1:nSteps
        state = yNum(iStep, :).';
        k1 = checkedOdeSlope(odeFun, t(iStep), state);
        k2 = checkedOdeSlope(odeFun, t(iStep + 1), state + h*k1);
        nextState = state + h*(k1 + k2)/2;
        if any(~isfinite(nextState))
            error('rk2Solve:NonfiniteState', '第 %d 步的数值解出现 NaN/Inf。', iStep);
        end
        yNum(iStep + 1, :) = nextState.';
    end
    meta = struct('stepSize', h, 'nSteps', nSteps, ...
        'method', '改进 Euler (RK2)', 'functionEvaluations', 2*nSteps);
end
function [t, yNum, h] = prepareOdeGrid(~,tSpan,y0,nSteps)
    nSteps=double(nSteps); h=(tSpan(2)-tSpan(1))/nSteps;
    t=linspace(tSpan(1),tSpan(2),nSteps+1).'; yNum=zeros(nSteps+1,numel(y0));
    yNum(1,:)=double(y0(:)).';
end
function slope = checkedOdeSlope(f,time,state)
    slope = f(time,state);
    slope=double(slope(:));
end
```

A.8 solvePoissonShooting.m

源文件: src/poisson/solvePoissonShooting.m。

```
function [x, uNum, meta] = solvePoissonShooting(nIntervals, sourceFun, boundaryValues,
xSpan)
    if nargin < 1 || isempty(nIntervals), nIntervals = 32; end
    if nargin < 2 || isempty(sourceFun), sourceFun = @(x) exp(-x.^2); end
    if nargin < 3 || isempty(boundaryValues), boundaryValues = [0, 0]; end
    if nargin < 4 || isempty(xSpan), xSpan = [-1, 1]; end
    validateattributes(nIntervals, {'numeric'}, {'scalar', 'integer', '>='}, 1);
    validateattributes(boundaryValues, {'numeric'}, {'vector', 'numel', 2, 'finite'});
    validateattributes(xSpan, {'numeric'}, {'vector', 'numel', 2, 'finite'});
    leftBoundary = boundaryValues(1);
    rightBoundary = boundaryValues(2);
    odeFun = @(xValue, state) [state(2); checkedSource(sourceFun, xValue)];
    [x, stateZeroSlope, rkMeta] = rk4Solve(odeFun, xSpan, [leftBoundary; 0],
nIntervals);
    intervalLength = xSpan(2) - xSpan(1);
    initialSlope = (rightBoundary - stateZeroSlope(end, 1)) / intervalLength;
    uNum = stateZeroSlope(:, 1) + initialSlope .* (x - xSpan(1));
    uNum(1) = leftBoundary;
    uNum(end) = rightBoundary;
    meta = struct();
    meta.stepSize = rkMeta.stepSize;
    meta.nIntervals = nIntervals;
    meta.initialSlope = initialSlope;
    meta.zeroSlopeEndValue = stateZeroSlope(end, 1);
    meta.functionEvaluations = rkMeta.functionEvaluations;
    meta.boundaryResidual = [uNum(1) - leftBoundary; uNum(end) - rightBoundary];
end
```

```

function sourceValue = checkedSource(sourceFun, xValue)
    sourceValue = sourceFun(xValue);
    sourceValue = double(sourceValue);
end

```

A.9 errorNorms.m

源文件: src/common/errorNorms.m。

```

function [maxError, l2Error, rmsError] = errorNorms(numericalValues, exactValues)
    validateattributes(numericalValues, {'numeric'}, {'nonempty', 'finite'});
    validateattributes(exactValues, {'numeric'}, {'nonempty', 'finite'});
    errorValues = numericalValues(:) - exactValues(:);
    maxError = max(abs(errorValues));
    l2Error = norm(errorValues, 2);
    rmsError = sqrt(mean(abs(errorValues).^2));
end

```

A.10 computeOrder.m

源文件: src/common/computeOrder.m。

```

function [pairwiseOrder, fittedOrder] = computeOrder(stepSizes, errorValues)
    validateattributes(stepSizes, {'numeric'}, {'vector', 'real', 'finite', 'positive'});
    validateattributes(errorValues, {'numeric'}, {'vector', 'real', 'finite', 'positive'});
    stepSizes = stepSizes(:);
    errorValues = errorValues(:);
    stepRatio = stepSizes(1:end-1) ./ stepSizes(2:end);
    errorRatio = errorValues(1:end-1) ./ errorValues(2:end);
    pairwiseOrder = log(errorRatio) ./ log(stepRatio);
    fitCoefficients = polyfit(log(stepSizes), log(errorValues), 1);
    fittedOrder = fitCoefficients(1);
end

```

附录 B 补充数据与诊断图

本附录列出空间误差的补充度量和计时波动。

表 B-1 有限差分补充误差与残差

N	加权离散 L2	代数残差	截断残差	条件数估计
8	3.2001e-03	1.4433e-15	1.0288e-02	32
16	7.9238e-04	3.3307e-15	2.5961e-03	128
32	1.9762e-04	2.3204e-14	6.5053e-04	512
64	4.9377e-05	1.0358e-13	1.6273e-04	2048
128	1.2342e-05	6.0252e-13	4.0688e-05	8192
256	3.0855e-06	1.6953e-12	1.0172e-05	32768

表 B-1 中的条件数由 `condest` 估计，用于描述离散矩阵的条件性。加权离散 L2 与 RMS 采用不同的归一化方式，数值含义也不同。

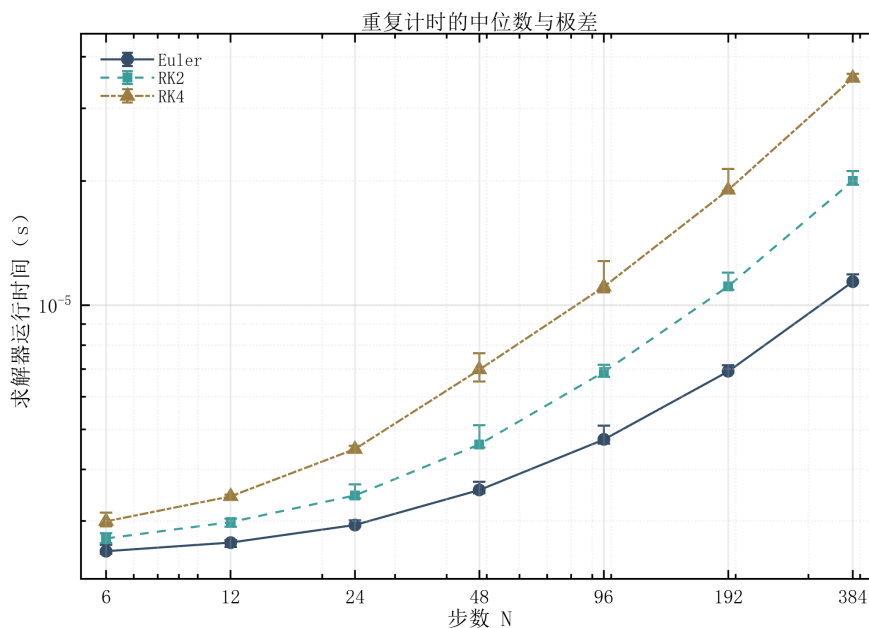


图 B-1 三方法重复计时的中位数及极值范围

图 B-1 的区间表示同一组合七次 `timeit` 估计的最小值和最大值，用于观察计时波动。原始文件同时记录每轮的方法执行顺序，可用于复核轮换安排。

附录 C 运行说明与结果索引

将 MATLAB 当前目录切换到项目根目录并运行 `runAll`，可依次生成两题基础实验、有限差分误差分析、Richardson 外推、打靶交叉验证、正式收敛实验、三方法比较和 RK4 补充图。程序会更新 `results` 和 `figures` 目录中的同名文件。

程序 C-1 实验入口与单独测试入口

```
runAll
```

```
addpath(genpath(fullfile(pwd, 'src')));
runAllTests();
```

`runAllTests` 是无返回值的独立测试入口，需要单独调用。测试内容包括误差工具、阶数计算、文件导出、有限差分残差、制造解、外推、符号判零、向量 RK4、方法调用次数和打靶收敛。测试用于检查程序性质，论文中的数值和图表则取自正式实验文件。

表 C-1 主要结果文件与对应生成程序

结果文件	生成程序	内容
<code>poisson_all_solutions.mat</code>	<code>runCoreExperiments.m</code>	六组有限差分节点结果
<code>poisson_analysis_summary.csv</code>	<code>runPoissonAnalysis.m</code>	误差、残差及条件数
<code>poisson_convergence.csv</code> / <code>ode_convergence.csv</code>	<code>runConvergenceExperiment.m</code>	两题七级正式收敛
<code>poisson_richardson_summary.csv</code>	<code>runRichardsonExperiment.m</code>	六组粗细网格外推
<code>ode_experiment.mat</code>	<code>runCoreExperiments.m</code>	四组 RK4 基础节点及符号检验
<code>ode_method_comparison.csv</code>	<code>runOdeMethodComparison.m</code>	三方法精度与计时汇总
<code>ode_method_timing_raw.csv</code>	<code>runOdeMethodComparison.m</code>	147 条原始计时估计
<code>poisson_shooting_raw.mat</code>	<code>runShootingExperiment.m</code>	打靶、差分与解析值
<code>poisson_shooting_analysis_summary.csv</code>	<code>runShootingAnalysis.m</code>	三组误差及收敛指标

论文表格按统一显示精度保留有效数字，CSV 文件保存更完整的数值。误差比和拟合阶均以原始数据计算。图中的理论参考线用于比较斜率，正文图注则以问题名称说明各图对象。